

Informe Técnico Avanzado: Manipulación de Paquetes con Scapy y Automatización de SSH con Paramiko

I. Introducción a las Herramientas Críticas de Ciberseguridad en Python

A. Contexto para el Futuro Analista de Ciberseguridad

La transición de estudiante a analista de ciberseguridad requiere un dominio profundo de las herramientas que permiten la interacción directa con la infraestructura de red y los sistemas remotos. Python se ha consolidado como el lenguaje preferido en este ámbito, actuando como la *lingua franca* para la automatización, el análisis de datos y la orquestación de la seguridad. Dentro del ecosistema de Python, las librerías Scapy y Paramiko son fundamentales para dos aspectos complementarios de la seguridad: la interacción a nivel de bit y byte con los protocolos de red (Scapy) y el control seguro de la infraestructura remota (Paramiko).⁵

Scapy, en su rol, permite a los ingenieros de red y seguridad entender exactamente qué capas tecnológicas están desarrollando o auditando, ya que ofrece la capacidad de forjar o decodificar paquetes de una amplia gama de protocolos, operando eficazmente en todas las capas relevantes del modelo TCP/IP (equivalente desde la Capa 2 hasta la Capa 7 del modelo OSI).⁷ Paramiko, por su parte, es esencial para la automatización de la gestión de infraestructuras de seguridad (SecOps), proporcionando una implementación robusta del protocolo SSHv2 para reemplazar métodos de acceso remoto obsoletos e inseguros, como Telnet o rsh. El dominio de ambas herramientas permite al futuro analista construir soluciones personalizadas para tareas que van desde el escaneo de vulnerabilidades hasta la respuesta a incidentes.

B. Nota de Ética Profesional: El Fuego de Prometeo en las Redes

Las capacidades ofrecidas por Scapy, en particular, lo posicionan como una herramienta de doble filo. Si bien es invaluable para tareas defensivas como el análisis de tráfico (*sniffing*), la detección de vulnerabilidades, y la realización de pruebas unitarias en protocolos³, también puede ser utilizado para fines ofensivos, incluyendo ataques de *spoofing*, *fuzzing* o inyección de tramas inválidas.³

Para cualquier profesional que aspire al rol de analista, es imperativo que el uso de estas librerías se adhiera a un estricto marco legal y ético. Esto implica limitar cualquier manipulación o inyección de paquetes a entornos de laboratorio controlados o a redes donde se posea consentimiento explícito y documentado (pruebas de penetración autorizadas).⁴ La forja y el envío de paquetes maliciosos o de gran volumen sin autorización previa se considera una actividad ilegal que puede resultar en la interrupción del servicio o en la explotación no ética de sistemas.

II. Scapy: Arquitectura, Manipulación y Forja de Paquetes

A. Fundamentos Arquitectónicos de Scapy

Scapy se define como un programa de manipulación de paquetes interactivo que permite enviar, diseccionar y forjar paquetes de red.³ Su arquitectura está diseñada para ofrecer una gran flexibilidad, permitiendo manejar la mayoría de las tareas clásicas de red, como el escaneo, el *tracerouting* y la inyección de paquetes, y a menudo se utiliza como un reemplazo directo para herramientas especializadas como *hping* o *arping*.³

El diseño de Scapy se centra en la facilidad para diseñar paquetes rápidamente. Funciona instanciando clases de protocolo (como IP, TCP, UDP, Ether) que el usuario puede apilar. Un aspecto que distingue a Scapy es que decodifica y presenta el paquete completo, no solo una interpretación de alto nivel (como "puerto abierto" o "cerrado").³ Esta capacidad de inspección a nivel de bit es vital para el análisis forense y la construcción de paquetes de ataque muy específicos.

B. Construcción y Sintaxis Esencial de Paquetes

1. Instanciación de Capas y Protocolos

La construcción de paquetes en Scapy comienza con la instanciación de las clases que representan los protocolos de red. Por ejemplo, `IP()` representa un encabezado de Internet Protocol. Para entender qué campos pueden ser manipulados dentro de una capa, se utiliza el comando `ls()`:

```
>>> from scapy.all import *
>>> ls(IP)
```

La función `ls(Protocolo)` lista todos los campos disponibles dentro de ese protocolo, junto con sus valores por defecto y tipos, lo cual es fundamental para personalizar el paquete.¹

2. El Operador de Apilamiento (/)

La sintaxis más distintiva de Scapy es el uso del operador de división, `/`, para simular el proceso de encapsulación de red. Este operador apila capas de protocolo de forma jerárquica: la capa más baja (más cercana al hardware, como Ether) se coloca primero, seguida por la capa de red (IP), la capa de transporte (TCP o UDP), y finalmente la capa de aplicación o los datos.

```
# Ejemplo de apilamiento TCP (Capa 2 / Capa 3 / Capa 4)
paquete_tcp = Ether()/IP(dst="192.168.1.1")/TCP(dport=80, flags="S")
```

Los valores de los atributos en las capas inferiores se transmiten y son utilizados por las capas superiores a menos que se sobrescriban explícitamente.²⁷

3. Campos por Defecto y Spoofing

Scapy está diseñado para la conveniencia al asignar *valores por defecto sensibles* a los campos no especificados. Por ejemplo, si no se especifica la dirección IP de origen, Scapy la elegirá basándose en la tabla de rutas del sistema operativo y la dirección de destino. También calcula automáticamente los *checksums*.²³

Sin embargo, para tareas de seguridad como el *spoofing* o la inyección de paquetes, es necesario sobrescribir estos valores. Por ejemplo, se puede forjar la dirección de origen (`src`), la dirección de destino (`dst`), o el Time-to-Live (`ttl`) para propósitos de evasión o

pruebas.²⁸ La capacidad de Scapy para inyectar un valor de IP de origen arbitrario, incluso si no es el real, es esencial para simular ataques.⁵

4. Inclusión de Carga Útil (Payload)

Para interactuar con la capa de aplicación o para enviar datos brutos, se puede apilar una cadena de texto o un objeto Raw() al final de la pila de protocolos. Esto es crucial cuando se simula el tráfico de una aplicación específica (como HTTP) o se prueban los límites de los *parsers* de protocolos.¹¹

```
# Paquete TCP con carga útil de aplicación (datos RAW)
p = IP(dst="192.168.1.1")/TCP(sport=22, dport=22)/"GET HTTP/1.0\r\n\r\n"
```

5. Visualización y Debugging (Análisis Forense)

Una vez que se construye un paquete, Scapy ofrece varias funciones para su inspección, cruciales para el análisis forense de paquetes:

- `paquete.summary()`: Ofrece un resumen conciso del paquete en una sola línea.¹
- `paquete.show()`: Muestra una vista desarrollada de todas las capas y los valores de sus campos.¹
- `hexdump(paquete)`: Proporciona un volcado hexadecimal del paquete, esencial para el análisis de bajo nivel y para verificar la exactitud de los encabezados forjados.¹

C. Funciones de Envío y Recepción de Paquetes

La elección de la función de envío adecuada depende de dos factores: si se requiere una respuesta del destino y la capa de red a la que se va a enviar el paquete (Capa 3, IP, o Capa 2, Ethernet).²⁹ Las funciones que operan en Capa 3 (`send`, `sr`) delegan la resolución de direcciones MAC al sistema operativo. Las funciones de Capa 2 (`sendp`, `srp`) utilizan sockets de nivel de enlace, permitiendo la forja explícita de direcciones MAC, lo cual es vital para ataques de red local.²⁶

Las funciones con capacidad de respuesta (`sr`, `srp`, `sr1`) retornan dos listas de paquetes: `ans` (los pares de solicitud y respuesta recibidos) y `unans` (los paquetes enviados que no obtuvieron respuesta).²⁹ La función `sr1` hace lo mismo que `sr` pero retorna solamente el primer paquete de respuesta emitido por el destinatario.⁴ El hecho de que Scapy devuelva el paquete de respuesta completo, en lugar de una simple conclusión (como hace Nmap), obliga

al analista a realizar un análisis forense detallado del comportamiento de la pila del destino.³

El siguiente cuadro resume las funciones esenciales de envío de paquetes:

Tabla 1: Funciones Esenciales de Envío y Recepción en Scapy

Función	Capa de Envío	Espera Respuesta	Uso Principal	Retorno
send(pkt)	Capa 3 (IP)	No	Envío simple de paquetes IP (e.g., inundación ICMP). ¹	Número de paquetes.
sendp(pkt)	Capa 2 (Ethernet)	No	Spoofing MAC, envío de tramas ARP/DHCP. ²	Número de paquetes.
sr(pkt)	Capa 3 (IP)	Sí	Escaneo de puertos TCP/UDP, análisis de red. ¹	(Respondidos, No respondidos).
srp(pkt)	Capa 2 (Ethernet)	Sí	Escaneo de red ARP, ataques de capa de enlace.	(Respondidos, No respondidos).
sr1(pkt)	Capa 3 (IP)	Sí	Envío rápido, retorna solo la primera respuesta. ³	Primer paquete de respuesta.

D. Casos de Uso Avanzados de Scapy en Ciberseguridad

1. Escaneo de Puertos y Servicios (Stealth Scan)

Scapy es ideal para realizar escaneos personalizados que los escáneres comerciales podrían no detectar. El *Stealth Scan* (escaneo SYN) es un ejemplo, donde el atacante envía un paquete TCP con el *flag* SYN (S) y espera una respuesta SYN-ACK (SA) para confirmar que el puerto está abierto, o un RST/ACK (RA) si está cerrado, sin completar el *three-way handshake*. La función `sr1()` es a menudo utilizada en este contexto ya que solo se necesita la primera respuesta para determinar el estado del puerto.³

```
# Escaneo SYN al puerto 80 del destino 192.168.1.11
paquete_syn = IP(dst="192.168.1.11")/TCP(dport=80, flags="S")
respuesta = sr1(paquete_syn, timeout=1, verbose=0)
# La respuesta debe ser analizada para buscar el flag 'SA' o 'RA'. [3, 1]
```

2. Ataques de Capa de Enlace (ARP Poisoning)

El Address Resolution Protocol (ARP) carece de mecanismos de seguridad, lo que lo hace vulnerable al envenenamiento de caché ARP, una técnica que permite a un atacante engañar a las víctimas para que asocien una dirección IP con una dirección MAC falsificada. Esta técnica es la base de los ataques Man-in-the-Middle (MITM) en redes locales.

Scapy permite la forja de respuestas ARP falsas (donde `op=2` significa "is-at" o respuesta). Para mantener el ataque activo o "persistente", los paquetes deben enviarse en un bucle continuo.

```
from scapy.all import Ether, ARP, sendp
# Forjar respuesta ARP
# op=2 (is-at), psrc (IP spoofed), pdst (IP target), hwsrc (MAC attacker)
spoof_pkt = Ether(dst="ff:ff:ff:ff:ff:ff")/ARP(op=2, psrc='IP_Gateway', pdst='IP_Victima',
hwsrc='MAC_Atacante')
sendp(spoof_pkt, loop=1, verbose=0)
```

3. Análisis de Tráfico y Forense (sniff)

La función `sniff()` transforma Scapy en un potente sniffer, capaz de capturar paquetes en tiempo real, aplicar filtros y ejecutar funciones de *callback* sobre cada paquete capturado.²⁶ Es una herramienta crítica para el análisis forense de la red, especialmente cuando se busca tráfico no cifrado que revele credenciales o contenido malicioso.³²

```
# Captura 100 paquetes en la interfaz eth0 filtrando por tráfico Telnet (puerto 23)
paquetes = sniff(filter="tcp port 23", count=100, iface="eth0")
# Los paquetes capturados pueden ser analizados para buscar credenciales en texto plano.
```

4. Fuzzing y Pruebas de Resistencia (Conexión a Buffer Overflow)

Dado que el estudiante se encuentra en el módulo de *Buffer Overflow* (BOF), la funcionalidad de *fuzzing* de Scapy es de especial interés. El *fuzzing* es una técnica que implica inyectar datos aleatorios, inesperados o malformados en los campos de un paquete, con el objetivo de provocar fallos en el software o servicio de destino.¹⁵

Cuando un programa de aplicación tiene un desbordamiento de búfer explotable por la red, a menudo se debe a que no valida correctamente la longitud o el formato de los campos de entrada. Scapy, mediante la función `fuzz()`, automatiza la generación de valores aleatorios (`RandNum`, `RandShort`, etc.) para los campos de encabezado.¹⁵ Un analista puede construir un paquete con una carga útil RAW excesivamente grande o con campos IP aleatorios para probar la resiliencia de la pila o de un servicio específico.

```
from scapy.all import IP, UDP, fuzz, send
# Envía un paquete UDP con campos IP aleatorios (fuzzed) continuamente
fuzzed_pkt = IP(dst="192.168.1.5") / fuzz(UDP())
send(fuzzed_pkt, loop=1, verbose=0)
```

La capacidad de Scapy para inyectar cargas útiles RAW en cualquier capa de la pila de protocolos²⁷ lo convierte en la herramienta principal para simular los vectores de ataque que conducen a un BOF de red. El analista puede diseñar payloads deliberadamente sobredimensionados, utilizando Scapy como el conducto de programación que conecta la teoría de la vulnerabilidad con la explotación práctica, superando las limitaciones de otras herramientas de prueba.

III. Paramiko: Automatización Segura de Conexiones SSH/SFTP

A. Introducción al Protocolo SSH y Paramiko

Paramiko es una implementación pura en Python del protocolo Secure Shell (SSHv2), diseñada para proporcionar funcionalidades robustas de cliente y servidor. En la ciberseguridad moderna, donde la automatización de tareas y la gestión remota son esenciales, Paramiko ofrece la capa de comunicación cifrada y segura (128 bits) que el protocolo SSH garantiza, reemplazando protocolos inseguros como Telnet o rsh.

El valor central de Paramiko reside en permitir a los analistas comunicarse y controlar dispositivos remotos (servidores, equipos de red) a través de scripts. Aunque existen abstracciones de más alto nivel (como Netmiko), el uso directo de Paramiko proporciona una comprensión fundamental de cómo se gestiona una sesión SSH, mejorando la capacidad general del analista para la programación de seguridad y la automatización.

B. Componentes Arquitectónicos Principales

La librería Paramiko se construye sobre varias clases clave que reflejan la estructura del protocolo SSH:

- **Transport:** Esta clase maneja la conexión de bajo nivel, la negociación de algoritmos criptográficos y la gestión de la sesión física.
- **SSHClient:** Es la clase de alto nivel, ideal para la mayoría de los casos de uso. Simplifica el proceso de autenticación, la carga de claves de host y el inicio de canales lógicos para la ejecución de comandos remotos.
- **Channel:** Representa una sesión lógica dentro de una conexión SSH establecida. Las sesiones de shell interactivas o la ejecución de comandos se realizan a través de canales.
- **SFTPClient:** Se utiliza para la transferencia segura de archivos. Esta clase se inicializa a partir de una conexión SSHClient establecida y ofrece métodos para las operaciones de SFTP.

C. Gestión de Conexiones y Autenticación

El proceso típico de conexión comienza creando una instancia de SSHClient, configurando la política de claves de host y llamando al método connect().

Autenticación con Llave Pública

La autenticación con llave pública es el método preferido en SecOps debido a su seguridad superior y facilidad de automatización, evitando el envío repetido de contraseñas. Paramiko soporta varios formatos de clave (RSA, DSS, Ed25519).

```
import paramiko
# Cargar la llave privada
clave_rsa = paramiko.RSAKey.from_private_key_file('/ruta/a/id_rsa')
cliente = paramiko.SSHClient()
# Conectar usando la clave, en lugar de una contraseña
cliente.connect(hostname='servidor_remoto', username='user', pkey=clave_rsa)
```

Políticas de Host Key (Seguridad Crítica)

Antes de conectarse, el analista debe definir una política para manejar las claves de host desconocidas. La clave de host es esencial para verificar la identidad del servidor y mitigar los ataques Man-in-the-Middle (MITM).

La verificación de la clave de host no debe ser tratada como una simple formalidad. La política utilizada define la postura de seguridad del script. La función `client.load_system_host_keys()` carga las claves conocidas del sistema (el equivalente al archivo `known_hosts` de OpenSSH).

La siguiente tabla detalla las políticas disponibles:

Tabla 2: Políticas de Manejo de Clave de Host en `paramiko.SSHClient`

Política	Clase Paramiko	Acción en Clave Desconocida	Riesgo de Seguridad
Rechazo Total	<code>paramiko.RejectPolicy()</code>	La conexión falla de inmediato si la clave no está en <code>known_hosts</code> .	Bajo (Seguridad Máxima).
Advertencia	<code>paramiko.WarningPolicy()</code>	Conecta, pero emite una advertencia si la	Moderado.

		clave es desconocida.	
Adición Automática	paramiko.AutoAddPolicy()	Agrega automáticamente la clave desconocida al known_hosts local y permite la conexión.	Alto (Vulnerable a MITM).

La utilización de AutoAddPolicy() es extremadamente peligrosa en producción o auditoría. Este método elimina la protección contra MITM, ya que cualquier atacante que intercepte el tráfico podría suplantar la identidad del servidor SSH legítimo y su clave se aceptaría automáticamente. Un analista de seguridad debe siempre utilizar RejectPolicy o, al menos, WarningPolicy junto con un archivo known_hosts gestionado de forma segura. Esto garantiza que la integridad criptográfica de la sesión SSH se mantenga.

D. Ejecución Remota de Comandos y Scripts (SecOps)

La función exec_command() es la utilidad principal para ejecutar comandos en el servidor remoto de forma no interactiva. Esta función devuelve tres objetos que representan los flujos de entrada, salida y error estándar (stdin, stdout, stderr).

```
# Conexión establecida a través de la instancia 'cliente'
ssh_stdin, ssh_stdout, ssh_stderr = cliente.exec_command("dir")
output = ssh_stdout.read().decode()
print(f"Resultado de comando remoto: {output}")
```

Comandos con Escalada de Privilegios

La ejecución de comandos que requieren escalada de privilegios, como sudo, puede ser un desafío con exec_command() debido a que espera una entrada interactiva (la contraseña de sudo). Si el comando sudo se ejecuta sin la bandera -S, el script se bloqueará.

Para solventar esto, es necesario utilizar la opción sudo -S, que obliga a sudo a leer la contraseña desde la entrada estándar (stdin). El analista debe escribir la contraseña inmediatamente después de enviar el comando a través del canal stdin.write().

Para flujos de trabajo más complejos que involucran múltiples comandos interactivos o scripts remotos que requieren varias entradas (como un script que llama a scp), es preferible abrir un canal SSH completo y simular una sesión de shell interactiva para gestionar el flujo de I/O de manera más controlada.

Automatización Multi-Servidor para SecOps

La arquitectura de SSHClient facilita la automatización masiva, un requisito clave para el SecOps moderno. Al encapsular la lógica de conexión y comando dentro de objetos Python, un analista puede iterar sobre una lista de cientos de hosts, ejecutando comandos estandarizados (por ejemplo, verificando logs de seguridad, desplegando parches o revisando configuraciones SSH). Esta capacidad de conectar a múltiples dispositivos simultáneamente (o secuencialmente) y manejar las respuestas de forma programática permite al analista escalar sus tareas de monitoreo y cumplimiento.

E. Transferencia Segura de Archivos (SFTP)

El Protocolo de Transferencia de Archivos SSH (SFTP) utiliza la conexión SSH segura para transferir datos, ofreciendo un canal cifrado para los archivos, a diferencia del FTP tradicional que transmite datos en texto plano. Paramiko maneja esto a través de la clase SFTPCClient.

El flujo de trabajo SFTP requiere que se abra el cliente SFTP a partir de una conexión SSH ya establecida, se realicen las operaciones y, finalmente, se cierren ambas conexiones.

```
# 1. Establecer conexión SSH (cliente ya conectado)
```

```
sftp = cliente.open_sftp()
```

```
# 2. Subir (Upload) un archivo local a la ruta remota
```

```
sftp.put('/ruta/local/archivo.log', '/ruta/remota/destino.log')
```

```
# 3. Descargar (Download) un archivo remoto
```

```
sftp.get('/var/log/syslog', '/tmp/syslog_capturado.log')
```

```
# 4. Cerrar la conexión SFTP
```

```
sftp.close()
```

El objeto SFTPCClient no solo maneja put() y get(), sino que también permite la gestión del sistema de archivos remoto, incluyendo funciones para listar directorios, crear directorios o cambiar permisos (chown).

Paramiko en la Respuesta a Incidentes (IR)

La velocidad es crucial durante una Respuesta a Incidentes (IR). La funcionalidad de SFTP, combinada con la automatización de Paramiko, es invaluable en este contexto. Cuando se detecta un compromiso, un analista puede programar Paramiko para conectarse rápidamente a los servidores afectados, ejecutar comandos de recolección de evidencia (por ejemplo, copiar archivos de log específicos o capturar *dumps* de memoria) y luego utilizar `SFTPClient.get()` para descargar de forma segura esta evidencia forense a un host de análisis centralizado. Esta automatización reduce el tiempo de contención y asegura que la evidencia se recoja de manera cifrada, maximizando la eficiencia de la respuesta.

IV. Mejores Prácticas, Seguridad Operacional e Integración

A. Gestión de Credenciales y Mínimo Privilegio (PoLP)

El uso de Paramiko y Scapy, al ser herramientas de alto privilegio para interactuar con la red y la infraestructura, exige la implementación de buenas prácticas de seguridad operativa. El mayor riesgo de seguridad es el almacenamiento inadecuado de secretos.

- **Evitar el Hardcoding:** Almacenar contraseñas, *passphrases* de claves o direcciones IP sensibles directamente en el código fuente de Python (hardcoding) es una práctica inaceptable. Si el código cae en manos equivocadas, todas las credenciales se comprometen.
- **Gestión de Secretos:** Los analistas deben utilizar soluciones profesionales de gestión de secretos (como HashiCorp Vault o variables de entorno seguras) para obtener dinámicamente las credenciales necesarias, asegurando que los secretos sean cifrados en reposo y rotados regularmente.
- **Principio de Mínimo Privilegio (PoLP):** Las cuentas de usuario SSH utilizadas por los scripts de automatización de Paramiko deben estar diseñadas siguiendo el PoLP. Deben tener solo los permisos necesarios para realizar la tarea automatizada (por ejemplo, si el script solo recolecta logs, solo debe tener permisos de lectura). Esto reduce la superficie de ataque y mitiga el riesgo de que un atacante, si compromete el script, pueda acceder a recursos críticos.

B. Integración de Scapy y Paramiko en un Flujo de Trabajo de Ciberseguridad

La verdadera potencia de estas librerías se manifiesta al integrarlas en flujos de trabajo automatizados. Paramiko proporciona el canal seguro para la orquestación remota, y Scapy proporciona la capacidad de prueba y análisis a nivel de paquete.

Escenario de Integración: Despliegue de Análisis Forense Remoto

Un analista sospecha de actividad maliciosa en un segmento de red. El objetivo es capturar el tráfico de red de ese segmento sin instalar software directamente ni exponer los datos de captura.

1. **Conexión Segura (Paramiko):** El analista utiliza SSHClient con autenticación de llave pública y RejectPolicy para establecer una conexión segura con un servidor de monitoreo o *jump box* dentro del segmento de red.
2. **Orquestación Remota (Paramiko):** Utilizando `exec_command()`, el analista ejecuta de forma remota un script de Python que contiene código Scapy. Este script remoto utiliza la función `sniff()` para capturar un número limitado de paquetes, aplicando filtros específicos para el tráfico bajo sospecha (por ejemplo, un puerto C2 conocido). El resultado de la captura se guarda en un archivo `.pcap` en el host remoto.
3. **Extracción Segura de Evidencia (SFTP):** Una vez que el script remoto ha terminado, el analista abre el cliente SFTPClient a través de la misma sesión SSH y utiliza el método `sftp.get()` para descargar el archivo `.pcap` de forma cifrada a su máquina local para un análisis forense más profundo.

Este flujo de trabajo demuestra una arquitectura de seguridad programática que es rápida, auditable y segura.

C. El Analista como Arquitecto de Seguridad

El dominio de Scapy y Paramiko marca un punto de inflexión en la carrera de un estudiante de ciberseguridad, transformándolo de un usuario de herramientas predefinidas a un arquitecto de soluciones de seguridad personalizadas.

La manipulación avanzada de paquetes que ofrece Scapy, especialmente la capacidad de apilar capas de protocolo de forma elástica y de inyectar datos RAW²⁷, permite al profesional ir más allá de los escaneos superficiales. Al diseñar paquetes con combinaciones de *flags* TCP ilegales o longitudes no estándar, el analista está probando la implementación subyacente de la pila de red del sistema operativo (*TCP Stack Auditing*), detectando comportamientos anómalos que herramientas más sencillas pasarían por alto.²⁰

De manera similar, el uso directo de Paramiko, en lugar de abstracciones, obliga al analista a

confrontar las dependencias criptográficas subyacentes (como la librería *cryptography* en C/Rust). Esto significa que el analista no solo entiende el código de Python, sino también cómo se gestiona el cifrado y la integridad de la sesión, lo cual es vital para el despliegue seguro en múltiples plataformas.

En resumen, la combinación de estas librerías proporciona la base técnica para realizar pruebas de penetración a bajo nivel, automatizar tareas críticas de SecOps, realizar análisis forenses de incidentes, y, lo que es más importante, construir herramientas que satisfagan necesidades de seguridad únicas, consolidando el perfil profesional del futuro analista de ciberseguridad.